

Implementation Plan — Phase 1

Project: court-admissible evidence chain for narcotics detection on Indian railways **Companion to:** `docs/evidence-report.md` (the research this is built on)

Each step below is **one GitHub issue and one pull request**. They are ordered so that every step only depends on steps above it. A peer reviewing a PR should be able to check the acceptance criteria without reading the whole repo.

How we work

```
main          ← protected, always demo-able
  feat/step-03-append-only-log
  feat/step-07-key-attestation
```

- One branch per step, named `feat/step-NN-short-name`.
- PR title = the step title. PR body = paste that step's **Acceptance** block and tick it.
- No PR merges without one peer approval and a green test run.
- If a step turns out to be wrong, open an issue titled `decision: <what changed>` and record it in

`docs/decisions.md`. Do not silently redesign — the decision log is what we show judges.

Repository layout

<code>/app-android/</code>	Kotlin handheld app
<code>/services/log/</code>	append-only log + Merkle tree
<code>/services/anchor/</code>	RFC 3161 timestamp client + scheduler
<code>/services/cert/</code>	Section 63 Schedule certificate generator
<code>/tools/verify/</code>	verifier CLI (the tamper demo)
<code>/dashboard/</code>	minimal web UI
<code>/sensor-adapter/</code>	hardware interface + mock implementation
<code>/docs/</code>	evidence report, decision log, demo script
<code>/tests/</code>	integration tests, fixtures

Phase A — The spine (no hardware needed)

Build and test this entirely on a laptop. If Phase A is not solid, nothing above it matters.

Step 1 — Repository skeleton and decision log

Do: create the folder structure above. Add `README.md`, `docs/decisions.md`, `.gitignore`, CI config that runs tests on every PR.

Files: `README.md`, `docs/decisions.md`, `.github/workflows/test.yml`

Acceptance

- ☐ CI runs and passes on an empty test suite
- ☐ `docs/decisions.md` has one entry: why we are not using Hyperledger Fabric

Reviewer checks: CI is actually wired to PRs, not just to `main`.

Step 2 — The record schema

Everything else serialises to this. Get it right before writing code that produces it.

Do: define one JSON schema covering all three record types, with a `type` discriminator.

Every record carries, at minimum:

FIELD	MEANING		
<code>record_id</code>	UUID, generated on device		
<code>type</code>	<code>trigger \</code>	<code>field_test</code>	<code>handoff</code>
<code>case_ref</code>	groups records belonging to one event		
<code>device_id</code>	from the attestation certificate, not user-entered		
<code>officer_id</code>	who is operating the device		
<code>claimed_time</code>	device clock at capture — untrusted, and labelled as such		
<code>seq</code>	monotonic counter value from the secure element		
<code>prev_hash</code>	hash of the previous record on this device		
<code>payload</code>	type-specific body		
<code>payload_hash</code>	SHA-256 over the canonicalised payload		
<code>signature</code>	over <code>payload_hash</code> + <code>prev_hash</code> + <code>seq</code> , by the hardware-backed key		

Files: `docs/schema/record.schema.json`, fixtures in `tests/fixtures/`

Acceptance

- ☐ Schema validates three example records, one of each type
- ☐ Canonicalisation is deterministic — same payload always produces the same hash, key order irrelevant
- ☐ `claimed_time` is documented in the schema as untrusted

Reviewer checks: run the canonicaliser twice on re-ordered JSON, confirm identical hashes.

Step 3 — Append-only log service

Do: Postgres table where each row stores `prev_hash`. Insert-only; a trigger blocks `UPDATE` and `DELETE` at the database level.

Files: `services/log/`, `migrations/001_records.sql`

Acceptance

- ☐ Inserting a record links it to the previous row's hash
- ☐ `UPDATE records SET ...` raises an error from the DB trigger
- ☐ A test that inserts 1,000 records and walks the chain end to end passes

Reviewer checks: try to update a row by hand in `psql`. It must fail.

Note for the demo: the trigger is defence in depth, not the security property. The chain is what

proves tampering. For the stage demo we will disable the trigger deliberately so we can edit a row

and let the verifier catch it.

Step 4 — Merkle history tree

Do: build a Merkle tree over the log. Produce inclusion proofs (this record is in the log) and consistency proofs (this log is an append-only extension of an earlier version).

Files: `services/log/merkle.py` (or `.ts`), `tests/test_merkle.py`

Acceptance

- ☐ Inclusion proof verifies for any record
- ☐ Consistency proof verifies between any two tree sizes
- ☐ A tampered record fails its inclusion proof

Reviewer checks: the proof size should grow logarithmically, not linearly. Print proof length for 10, 100, 1,000 records and eyeball it.

Step 5 — Verifier CLI, version 1

The tamper demo lives here. Build it early; it is how you test everything else.

Do: `verify --db <conn>` re-walks the whole chain and reports the first broken link.

Files: `tools/verify/`

Acceptance

- ☐ Clean log → exits 0, prints record count and tree head
- ☐ Edit one row in Postgres → exits non-zero, names the exact `record_id` where the chain breaks
- ☐ Output is readable by a non-engineer standing three metres from a projector

Reviewer checks: actually break a record and run it. Screenshot the output into the PR.

Phase B — The device

Step 6 — Sensor adapter with a mock

This answers "read input from the hardware" without owning the hardware.

We do not have an IMS detector and will not fabricate one. Define the interface now so a real device can be dropped in later, and ship a mock behind it.

Do: one interface, two implementations.

```
SensorAdapter
    read() -> SensorReading { raw_value, tier, sensor_type, sensor_id, taken_at }

MockSensorAdapter      - returns scripted readings from a config file, for the demo
SerialSensorAdapter    - reads newline-delimited JSON over USB/serial, for when hardware exists
```

Files: `sensor-adapter/`, `sensor-adapter/mock_readings.yaml`

Acceptance

- ☐ Swapping implementations requires changing one config line, no app code
- ☐ Mock readings are loaded from a file, not hard-coded
- ☐ Every reading is labelled with `sensor_type` so a record can never imply a capability we lack

Reviewer checks: the mock must be obviously a mock in the record output. We are not passing off simulated readings as real ones.

Step 7 — Hardware-backed key and attestation

Do: generate a key in Android Keystore, StrongBox where available. Capture the attestation certificate chain, including Verified Boot state.

Files: `app-android/crypto/KeyManager.kt`, `services/log/attestation.py`

Acceptance

- ☐ Key is generated with `setAttestationChallenge` and is non-exportable
- ☐ App reads and stores the attestation certificate chain
- ☐ Server **rejects** a record whose attestation chain does not verify
- ☐ The record shows whether the key is in StrongBox or TEE — do not claim StrongBox if it is TEE

Reviewer checks: run on a real handset, not the emulator. Emulator attestation is not evidence of anything. Paste the security level from the certificate into the PR.

Step 8 — On-device hash chain and monotonic counter

Do: each new record on a device links to the previous one. Use a secure-element monotonic counter for `seq` so the device cannot silently reorder or re-issue.

Files: `app-android/chain/LocalChain.kt`

Acceptance

- ☐ Records created offline form an unbroken local chain
- ☐ `seq` strictly increases and survives app restart
- ☐ Deleting the app's data and reinstalling does **not** let `seq` restart silently — the server

detects the discontinuity

Reviewer checks: kill the app mid-capture; confirm the chain is still valid on restart.

Step 9 — Offline queue

Do: records persist locally with a status, and survive being offline for hours.

captured → queued → pushed → anchored

Files: `app-android/queue/`

Acceptance

- ☐ Aeroplane mode: capture 20 records, force-quit, reopen — all 20 still queued, chain intact
- ☐ Status is visible in the UI so an officer knows what has not yet reached the server
- ☐ Queue survives a device reboot

Step 10 — Push to server

Do: on connectivity, send queued records in order. Server validates signature, attestation, and chain linkage before accepting.

Files: `app-android/sync/` , `services/log/ingest.py`

Acceptance

- ☐ Records arrive in `seq` order; out-of-order arrival is rejected with a clear error
- ☐ Replaying an already-pushed record is rejected as a duplicate
- ☐ Interrupting the push halfway leaves the queue consistent, and a retry completes it

Phase C — Time

This is the part you asked about, and it is the hardest idea in the project. Read this section before coding it.

How the timestamp actually works

A phone in aeroplane mode cannot know the time. Its clock can be wrong, and can be set wrong deliberately. So we never claim to prove when a record was made. We prove a **bounded interval**, and we make that interval small.

On the device, offline:

1. Capture the record. Write the device clock into `claimed_time` — labelled untrusted.
2. Hash the record.
3. Sign the hash with the hardware-backed key, together with `prev_hash` and `seq`.
4. Append to the local chain. **The chain proves ORDER without needing any clock.**

On reconnect:

1. Push records to the server (Step 10).
2. Server takes the current Merkle tree head and sends *that hash only* to an RFC 3161 timestamp authority. The record itself never leaves — a TSA only ever sees a hash.

1. TSA returns a signed token: *this hash existed at time T_{anchor}* .
2. Store the token against the tree head.

What you can now prove for any record:

T_previous_anchor ≤ record was created ≤ T_this_anchor

The record is provably *after* the previous anchor (it is not in that earlier tree) and provably *before* this anchor (it is in this tree). Its position relative to other records in the same window is fixed by the local hash chain.

So the honest claim is: "created between 14:02 and 14:19, in this order relative to these other records."
Never "created at 14:07."

Anchor often. The interval width equals the gap between anchors. Anchoring once per shift gives you an eight-hour window. Anchoring at every connectivity window gives you minutes. That is the whole reason the cadence matters, and it costs almost nothing — one small HTTP request per anchor, regardless of how many records it covers.

Step 11 — RFC 3161 client

Do: server-side client that submits a hash and stores the returned token.

Files: `services/anchor/tsa_client.py`

Acceptance

- ☐ Submits only a hash, never record content — assert this in a test
- ☐ Stores the token and verifies its signature against the TSA certificate
- ☐ Handles TSA unavailable without losing the record — retry queue

Reviewer checks: packet-capture or log the outbound request and confirm no payload leaks.

Step 12 — Anchor scheduler

Do: anchor the tree head on every connectivity window, and at minimum every N minutes.

Files: `services/anchor/scheduler.py`

Acceptance

- ☐ Anchors after every successful push batch
 - ☐ Records the anchor interval so it can be reported
 - ☐ Never anchors an unchanged tree head twice
-

Step 13 — Interval computation and display

Do: for any record, compute and show `[T_previous_anchor, T_this_anchor]`.

Files: `services/log/interval.py`, surfaced in the dashboard and the verifier

Acceptance

- ☐ Every anchored record displays its bounded interval
- ☐ Unanchored records display "**not yet anchored — time unproven**", not a claimed time
- ☐ The UI never presents `claimed_time` as if it were proven

Reviewer checks: this is the single most important honesty property in the system. Look for any place the interface shows a bare timestamp without its qualifier.

Step 14 — Second timestamp authority

Do: anchor to two independent TSAs in different trust domains. Compare.

Files: `services/anchor/` (extend)

Acceptance

- ☐ Both tokens stored per anchor
- ☐ Divergence beyond a configured tolerance raises a visible alert
- ☐ System degrades gracefully to one TSA, and says so

Why: our own argument against a single database administrator was that one party could rewrite

history. Anchoring to a single TSA would reintroduce a single trusted party. Two independent

authorities is the answer to that objection, and a reviewer will ask.

Phase D — The three records

Build in this order. Each is independently demo-able.

Step 15 — Record type: field test

The record a High Court has effectively specified. Build this one first — if the project has to be cut, this is what survives.

Do: capture form with **controlled vocabularies, not free text**.

FIELD	INPUT TYPE
Reagent / kit type	dropdown
Manufacturer	dropdown
Lot number	text
Expiry date	date picker, warns if expired
Observed colour	dropdown from a fixed list — never free text
Reference table used	dropdown
Ambient temperature	number, optional
Operator	from signed-in identity
Result status	fixed: <code>presumptive positive</code> / <code>presumptive negative</code> / <code>inconclusive</code>

Acceptance

- ☐ The rendered record prints "**Presumptive — not a chemical identification**" on its face
- ☐ Colour cannot be entered as free text
- ☐ An expired lot produces a visible warning that is carried into the record
- ☐ No screen anywhere says "confirmed", "identified" or "detected drug"

Step 16 — Record type: s.43 trigger

Do: capture what caused the stop — sensor output and tier from the adapter, place, device identity, and what the officer did next.

Acceptance

- ☐ Auto-populates from the sensor adapter; the officer does not type the reading
- ☐ Records the outcome — searched / not searched — even when nothing was found
- ☐ Links to the field-test record when one follows

Recording the *negatives* is what makes this data worth anything later. A log of only the hits

cannot tell anyone whether the programme works.

Step 17 — Record type: RPF → GRP handoff

The seam. Two devices, two officers, two signatures over one transfer.

Do: transferring officer creates the handoff record; receiving officer signs a receipt against the same `case_ref`.

Acceptance

- ☐ Both signatures verify independently
- ☐ Sample count and seal state are carried across and shown on both sides
- ☐ Receipt cannot be created without the transfer record existing
- ☐ Verifier shows the two-party link

Phase E — Court output

Step 18 — Section 63 Schedule certificate

Do: generate the Schedule certificate as a PDF, auto-filling what the system holds.

Acceptance

- ☐ Part A fields auto-filled: device type, make and model, serial, hash value, hash algorithm
- ☐ **SHA-256 ticked by default** — never MD5, even though the form offers it
- ☐ Part B (expert) left blank for signature — we do not sign as the expert
- ☐ Hash on the certificate matches the record hash, verified by a test

Step 19 — Verifier web view and tamper demo

Do: browser view of the verifier for the stage demo.

Acceptance

- ☐ Green for a clean chain, red naming the broken record

- ☐ Runs against a live database, not a fixture
- ☐ Rehearsed twice on a machine that is not the developer's laptop

Phase F — Surface and evidence

Step 20 — Minimal dashboard

Case list, one case view, handoff status, certificate download. **Nothing else.** No map, no heatmap, no live alert feed — those belong to the parked phase-2 work.

Acceptance

- ☐ A stranger can find a case and export its certificate without being shown how

Step 21 — Field-note instrumentation

Do: log the numbers nobody else in this field has published — time to complete each record type, connectivity window frequency and duration, anchor interval distribution, fields left blank.

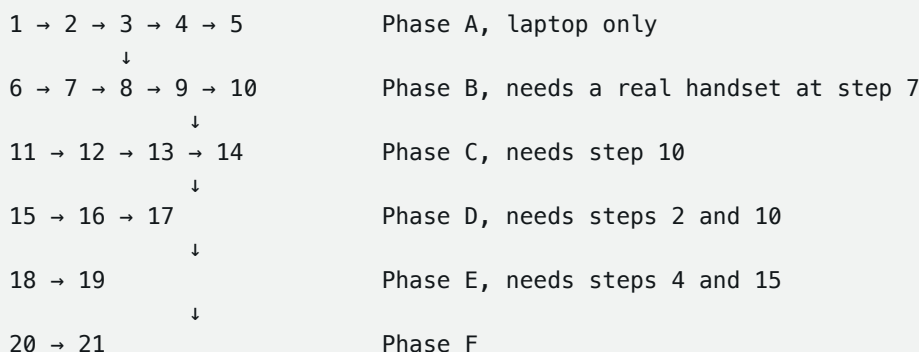
Acceptance

- ☐ Metrics export as CSV
- ☐ Two weeks of use produces a table you can put in the pitch

The blockchain-evidence literature has dozens of designs and almost no deployment reports. Real

numbers from one station would be a more original contribution than any architecture.

Dependency order



Parallelisable: Steps 6 and 7 can run alongside Phase A. Step 18 only needs the schema and the tree, so it can start during Phase D.

Definition of done, for every step

1. Branch merged to `main` with one peer approval
 2. Tests pass in CI
 3. Acceptance boxes ticked in the PR body
 4. Any design change recorded in `docs/decisions.md`
 5. `main` still runs the demo end to end
-

Things we are deliberately not building

Record these in the README so a reviewer does not "helpfully" add them back:

- **Thermal imaging for concealment** — rail-specific peer-reviewed work says thermal cannot detect concealed objects; that requires millimetre-wave sensing.
- **Behavioural anomaly CCTV** — four meta-analyses across 400+ studies put behavioural deception detection at roughly chance, and no video benchmark contains a concealed-contraband class.
- **Predictive patrol routing** — published trials ran at police-division scale; one station cannot train such a model, and training on our own detections creates a feedback loop.
- **Hyperledger Fabric** — its default ordering service is crash-fault-tolerant, not Byzantine, so a three-node network is defeated by two colluding parties. India's own eSakshya uses hash plus immutable storage, not a ledger.

Each of these was tested against the evidence and failed. Removing them is a finding, not a gap.